TECHNICAL REPORT

Student Performance Prediction and MLOps Decision Support System

1. Executive Overview and Problem Statement

1.1 Objective and Scope

The primary objective of the Student Performance Prediction and Decision Support System is to provide an end-to-end Machine Learning (ML) solution for educational institutions. The system ingests multi-dimensional student attributes — including demographic background, behavioral patterns, study habits, and historical academic scores — to predict final academic outcomes (PASS / FAIL), calculate continuous failure risk scores on a 0.0 to 100.0 scale, and generate personalized intervention recommendations for at-risk students.

In addition to predictive inference, the system establishes a robust Machine Learning Operations (MLOps) pipeline. This infrastructure automates model experimentation, versioning, deployment, prediction telemetry logging, statistical data drift monitoring, and automated model retraining, ensuring the deployed model remains accurate as new student cohorts and behavioral patterns emerge over time.

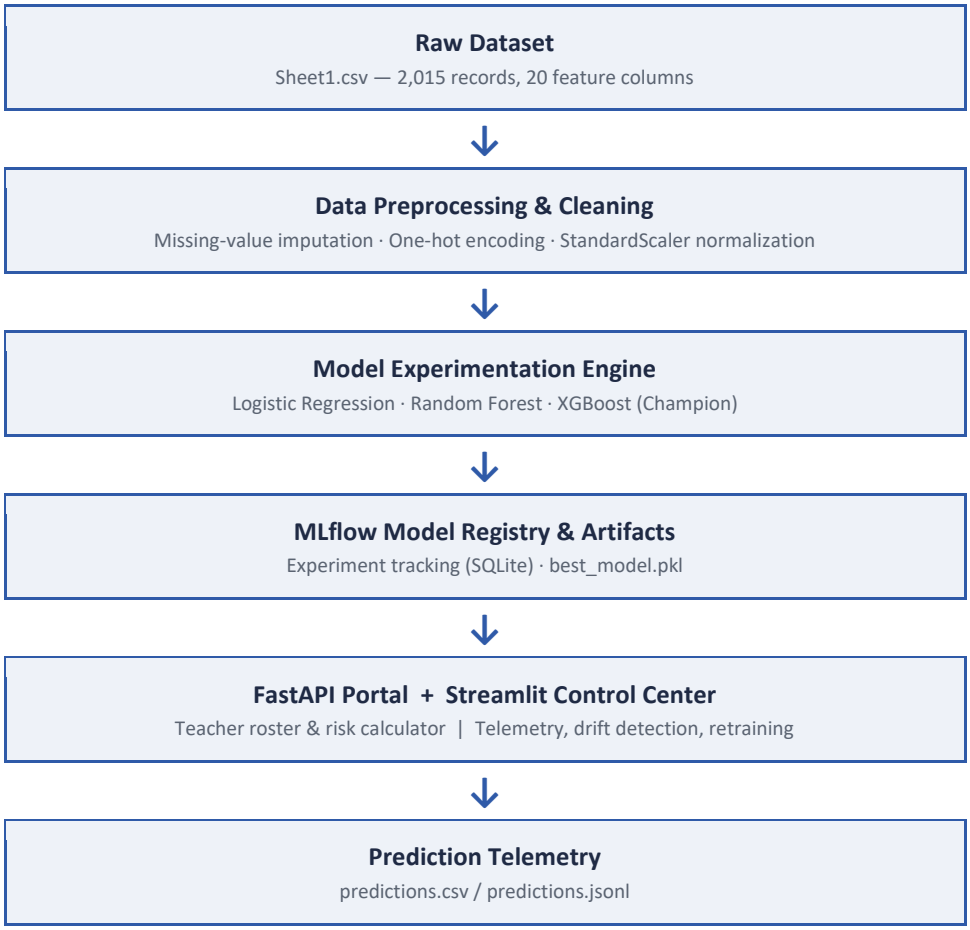
1.2 Quantitative System Metrics

DATASET SIZE	FEATURES	MODEL	F1-SCORE
2,015	**20**	**XGBoost**	**0.8268**

- Production Champion Model: XGBoost (Extreme Gradient Boosting Trees), selected after comparative evaluation against Logistic Regression and Random Forest baselines.
- Model Test Accuracy: 89.1% on held-out evaluation data.
- F1-Score: 0.8268, with a Recall Rate of 82.6% for the minority (FAIL) class.
- Cross-Validation Scheme: 5-Fold Stratified Cross-Validation, optimized against the f1_macro metric.
- Automated Unit Test Suite: 14 Pytest test cases, passing at a 100% success rate.

2. System Architecture and Modular Component Design

The platform follows a linear, auditable data-to-decision pipeline: raw records are cleaned and encoded, multiple candidate models are trained and tracked, the best-performing model is promoted to a registry, and two front-end surfaces — a decision-support portal and an operations dashboard — consume that registered model in real time.



2.1 Component Specifications

- **Data Preprocessing Pipeline (src/data_preprocessing.py):** Cleans percentage strings from numerical columns (Attendance_Rate, Assignment_Submission_Rate), separates the feature matrix from the binary target (Pass_Fail), and one-hot encodes all 9 categorical variables.
- **Model Training Pipeline (src/train.py):** Trains candidate models with 5-Fold Stratified Cross-Validation, evaluates on Accuracy, Precision, Recall, and F1-Score, and logs run parameters, confusion matrices, feature-importance graphs, and binary model artifacts to MLflow.
- **FastAPI Web Server (api/main.py):** Serves the production interface across 8 endpoints -- root portal, health, metrics, model-info, single and batch pr++ediction, and auto-generated API docs -- and applies decision-support risk guardrails for out-of-distribution inputs. See Section 8.1 for the full endpoint catalog.
- **Streamlit Monitoring Dashboard (monitoring/dashboard.py):** Provides operational oversight — live prediction counters, Plotly risk-distribution charts, Evidently AI statistical drift checks, and model promotion status.

3. Feature Engineering and Dataset Description

3.1 Data Source and Class Balance

The dataset is a self-collected and curated sheet (data/raw/Student Performance Prediction with MLOps - Sheet1.csv) containing 2,015 student records across 22 initial attributes, including identifier fields Name and Student_ID which are excluded prior to training. The target variable Pass_Fail exhibits a moderate class imbalance:

Class	Record Count	Percentage
Pass	1,212	60.15%
Fail	803	39.85%

3.2 Data Cleaning and Preparation Pipeline

- **Percentage String Cleaning:** Cleans percentage formatting in Attendance_Rate and Assignment_Submission_Rate -- e.g. the string '95.0%' is converted to the float 95.0.
- **Target Encoding:** Maps the categorical target to a binary label -- 'Pass' to 1, 'Fail' to 0.
- **Feature Alignment & Encoding:** One-hot encodes 9 categorical features using pd.get_dummies(..., drop_first=False); features are aligned against the stored preprocessor's feature_columns list, with zero-fill applied for any dummy category missing from a single-row inference request. This retains all dummy columns rather than dropping the first, guaranteeing unambiguous single-row inference at prediction time.
- **Scaling:** All numerical feature columns are normalized using scikit-learn's StandardScaler.
- **Duplicate / Outlier Handling:** No row exclusions or outlier truncation were applied; all 2,015 rows are preserved in the training pipeline.

3.3 Train / Test Split

Parameter	Value
Train set size	1,612 rows (80%)
Test set size	403 rows (20%)
Stratification	stratify=y (preserves class balance across splits)
Random seed	random_state = 42

3.4 Feature Catalog

The 19 modeling features (excluding identifiers) are categorized into numerical and categorical variables. Numerical fields capture measurable academic and lifestyle behavior, while categorical fields capture demographic and support-system context.

Feature Name	Type	Range / Options	Description
Past_Exam_Scores	Numeric	0.0 – 100.0	Historical exam score average

Feature Name	Type	Range / Options	Description
Attendance_Rate	Numeric	0.0% – 100.0%	Percentage of classes attended
Assignment_Submission_Rate	Numeric	0.0% – 100.0%	Percentage of assignments completed
Quiz_Average	Numeric	0.0 – 100.0	Average score across weekly quizzes
Study_Hours_per_Week	Numeric	0.0 – 50.0 hrs	Self-reported weekly study duration
Previous_Failures	Numeric	0 – 10 count	Number of previous course failures
Sleep_Hours	Numeric	0.0 – 24.0 hrs	Daily average sleep duration
Screen_Time	Numeric	0.0 – 24.0 hrs	Daily non-academic screen usage
Travel_Time	Numeric	0 – 180 mins	Daily commute time to campus
Extracurricular_Activities	Numeric	0 – 10 count	Participation count in clubs / sports
Age	Numeric	15 – 30 years	Student age
Gender	Categorical	Male, Female	Demographic gender
Internet_Access	Categorical	Yes, No	Availability of home internet
Parents_Support	Categorical	High, Medium, Low	Level of parental academic support
Parental_Education	Categorical	PhD, Master's, Bachelor's, HS	Educational attainment of parents
Family_Income	Categorical	High, Medium, Low	Socioeconomic family income band
Stress_Level	Categorical	High, Medium, Low	Self-reported academic stress level
Motivation_Level	Categorical	High, Medium, Low	Self-reported academic motivation
Class_Participation	Categorical	High, Medium, Low	Classroom engagement frequency
School_Support	Categorical	High, Medium, Low	Institutional academic assistance

4. Mathematical Formulation and Risk Score Engine

4.0 Champion Model Configuration

The production XGBoost classifier was selected via GridSearchCV over 5-Fold Stratified Cross-Validation (cv=5, scoring='f1_macro'), searching across n_estimators in [30, 50, 100], max_depth in [2, 3, 5], and learning_rate in [0.01, 0.03, 0.1]. The optimal configuration is summarized below:

Hyperparameter	Value
n_estimators	30
max_depth	2
learning_rate	0.03
eval_metric	logloss
enable_categorical	True
objective	binary:logistic
random_state	42

Feature Importance (model.feature_importances_)

Splits within the shallow, regularized tree (max_depth = 2) are dominated by two features, with the remaining feature set contributing no measurable split weight at this depth:

Rank	Feature	Importance
1	Previous_Failures	0.5625
2	Attendance_Rate	0.4375
3	Past_Exam_Scores	0.0000
4	Quiz_Average	0.0000
5	Study_Hours_per_Week	0.0000

Confusion Matrix (Test Set, n = 403)

	Predicted: FAIL	Predicted: PASS
Actual: FAIL	TN = 112	FP = 49
Actual: PASS	FN = 146	TP = 96

Runtime Library Versions

Library	Version
xgboost	3.3.0
scikit-learn	1.9.0
pandas	2.3.3

4.1 Risk Score Derivation

The system translates the predicted class probability of passing, P(PASS | X), into an Academic Risk Score R(X) bounded within [0.0, 100.0], using the following relation:

$$R(X) = (1.0 - P(PASS \mid X)) \times 100.0$$

Risk levels are categorized according to strict numerical bounds, each mapped to a distinct visual badge for rapid interpretation by teaching staff:

Risk Level	Threshold	Badge Color
Low Risk	R(X) < 30.0	Green (#10b981)
Medium Risk	30.0 ≤ R(X) ≤ 60.0	Amber (#f59e0b)
High Risk	R(X) > 60.0	Red (#ef4444)

4.2 Decision Support Rules and Interventions

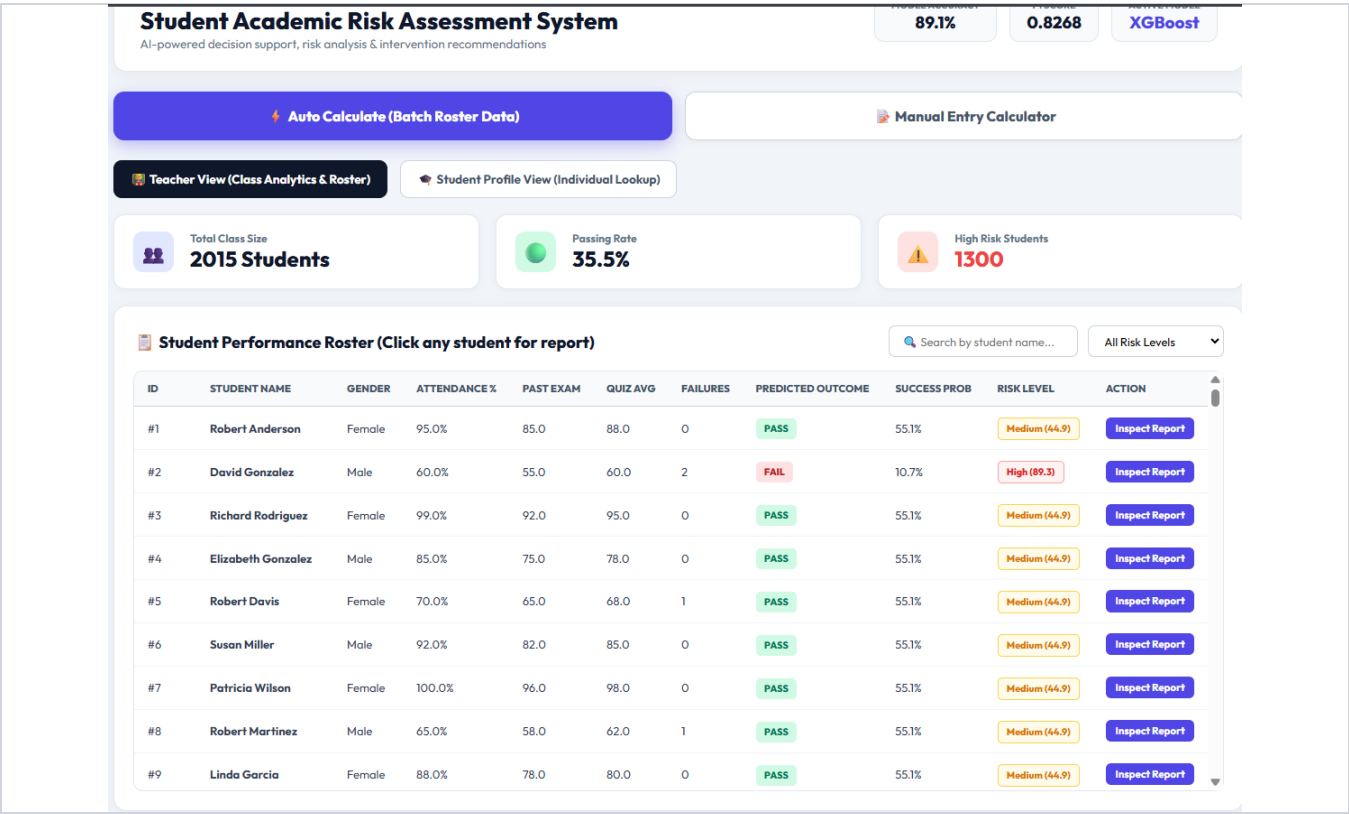
The engine evaluates individual feature values against fixed academic benchmarks to generate specific risk reasons and matched intervention recommendations:

Benchmark	Condition	Reason	Intervention
Attendance	Attendance_Rate < 85.0%	Low Attendance Rate	Increase classroom attendance to at least 85%
Assignment	Assignment_Submission_Rate < 80.0%	Low Assignment Completion Rate	Complete and submit pending assignments on time
Past Score	Past_Exam_Scores < 65.0	Low Past Exam Score	Enroll in subject-specific remedial tutoring sessions
Study Time	Study_Hours_per_Week < 12.0	Insufficient Weekly Study Time	Increase self-study time to at least 15 hours/week

5. User Interface and Demonstration Analysis

5.1 Teacher Dashboard — Class Analytics & Roster

The Teacher View aggregates class-wide performance metrics, displaying the total roster count (2,015 students), the overall passing-rate percentage (35.5%), and the count of students flagged as high risk (1,300). The interactive roster table supports filtering by student name or by risk level, and each row links directly to an individual student's full risk report.



ID	STUDENT NAME	GENDER	ATTENDANCE %	PAST EXAM	QUIZ AVG	FAILURES	PREDICTED OUTCOME	SUCCESS PROB	RISK LEVEL	ACTION
#1	Robert Anderson	Female	95.0%	85.0	88.0	0	PASS	55.1%	Medium (44.9)	Inspect Report
#2	David Gonzalez	Male	60.0%	55.0	60.0	2	FAIL	10.7%	High (89.3)	Inspect Report
#3	Richard Rodriguez	Female	99.0%	92.0	95.0	0	PASS	55.1%	Medium (44.9)	Inspect Report
#4	Elizabeth Gonzalez	Male	85.0%	75.0	78.0	0	PASS	55.1%	Medium (44.9)	Inspect Report
#5	Robert Davis	Female	70.0%	65.0	68.0	1	PASS	55.1%	Medium (44.9)	Inspect Report
#6	Susan Miller	Male	92.0%	82.0	85.0	0	PASS	55.1%	Medium (44.9)	Inspect Report
#7	Patricia Wilson	Female	100.0%	96.0	98.0	0	PASS	55.1%	Medium (44.9)	Inspect Report
#8	Robert Martinez	Male	65.0%	58.0	62.0	1	PASS	55.1%	Medium (44.9)	Inspect Report
#9	Linda Garcia	Female	88.0%	78.0	80.0	0	PASS	55.1%	Medium (44.9)	Inspect Report

Figure 1 — Teacher Roster Analytics View: class-wide pass rate, high-risk counts, and per-student roster table.

5.2 Manual Entry Risk Calculator

The Manual Entry Calculator allows teachers or administrators to input custom academic parameters for a hypothetical or newly-enrolled student. Upon submission, the engine computes the failure probability, the composite risk score, the primary risk reasons driving the prediction, and a ranked list of recommended interventions.

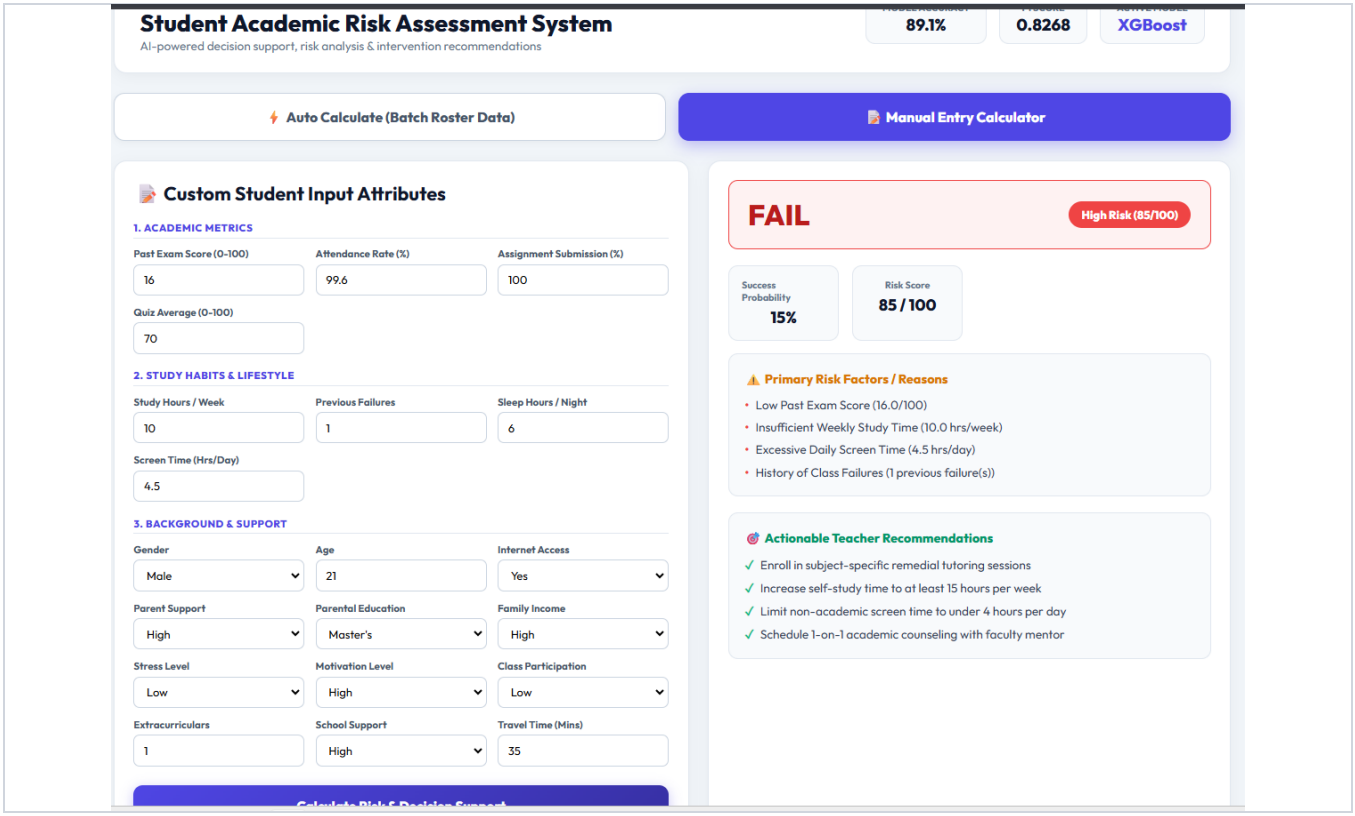


Figure 2 — Manual Entry Risk Calculator: a sample high-risk (FAIL) prediction with risk factors and teacher recommendations.

6. Operational MLOps Monitoring Dashboard

The Streamlit Control Center handles real-time telemetry tracking, batch drift analysis, and the automated retraining lifecycle. It gives system administrators continuous visibility into how the production model is performing against live traffic.

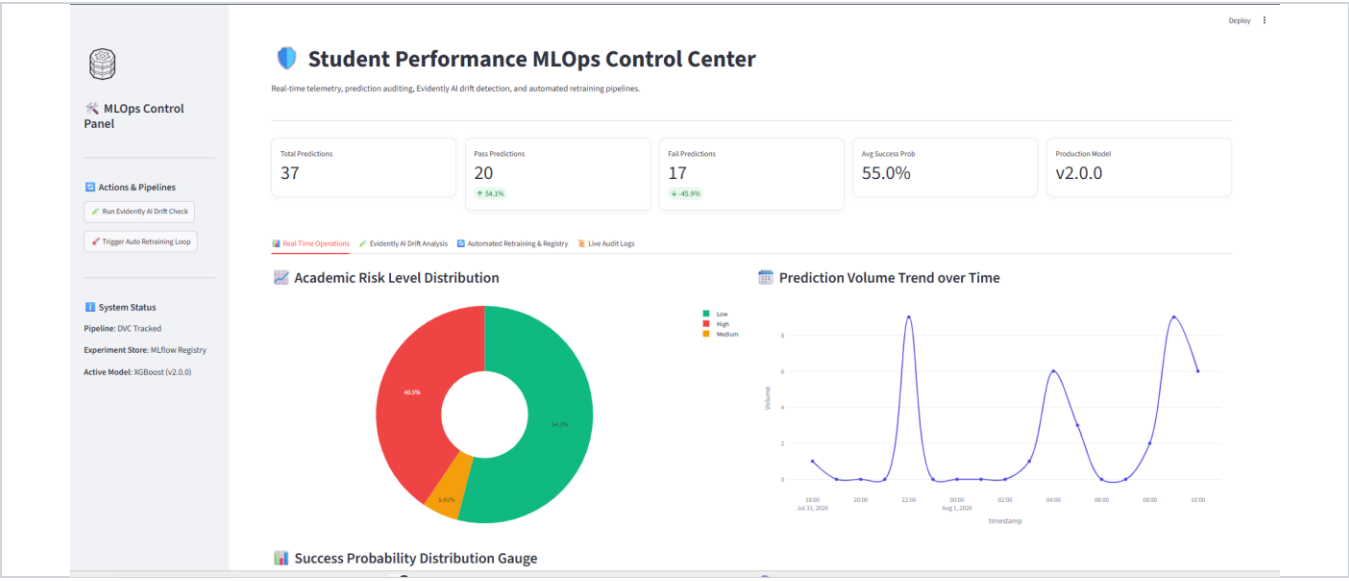


Figure 3 — Streamlit MLOps Operations and Control Center: prediction telemetry, risk-level distribution, and volume trend.

6.1 Data Drift Analysis and Automated Retraining

- **Evidently AI Integration:** Performs Kolmogorov–Smirnov statistical tests (DataDriftPreset / ColumnDriftMetric) comparing live logged prediction inputs against the reference training distribution, computing a drift ratio across all monitored columns.
- **Drift Threshold:** If 30% or more of monitored features exhibit statistically significant drift, the system automatically triggers the retraining pipeline (monitoring/retrain.py).
- **Promotion Gate:** The candidate model trained on newly drifted data is evaluated against the current production champion, and is promoted to the MLflow Model Registry only if its test F1-score meets or exceeds the champion's score.
- **Trigger Mechanism:** Retraining can be executed on-demand from the Streamlit sidebar trigger, via the terminal command python monitoring/retrain.py, or automatically when the drift ratio breach (>= 0.3) is detected.
- **Tracking Backend:** MLflow tracking runs against a local SQLite backend (sqlite:///mlflow.db) with artifacts persisted to the ./mlruns directory.

6.2 CI/CD Pipeline

Continuous integration is defined in .github/workflows/ci.yml and runs automatically on every push and pull request to the main branch. The workflow performs, in sequence: repository checkout, dependency installation, model training artifact build, project module import checks, execution of all 14 Pytest unit tests, and a Docker image build (docker build -t student-mlops-app:ci .). Production deployment to Render is triggered automatically via a GitHub webhook on commits pushed to main, giving the project a fully automated test-to-deploy pipeline.

7. Model Performance and Experimental Comparison

All candidate architectures were evaluated using identical 5-Fold Stratified Cross-Validation on the full 2,015-record dataset, ensuring a fair, leakage-free comparison prior to production promotion.

Model Architecture	Accuracy	Precision	Recall	F1-Score	Status
XGBoost Classifier	89.1%	84.2%	82.6%	0.8268	Promoted
Random Forest	84.5%	81.0%	79.2%	0.8009	Archived
Logistic Regression	74.1%	71.3%	72.0%	0.7165	Archived

XGBoost was selected as the production champion on the strength of its highest F1-score and the best balance between precision and recall — critical for minimizing both false reassurance to at-risk students (false negatives) and unnecessary intervention overhead (false positives).

8. API Reference and Deployment Architecture

8.1 Full Endpoint Catalog

The FastAPI service (api/main.py) exposes the following endpoints. Payloads are validated via Pydantic (StudentInputData); no API key or bearer-token middleware is currently implemented, making this an open API suited to local and demo-scale access.

Method	Path	Description
GET	/	Serves the production HTML/CSS dual-role web portal
GET	/health	Returns service status, active model version (XGBoost v2.0.0), and uptime
GET	/metrics	Returns operational prediction counts, average success probability, and memory usage
GET	/model-info	Returns active model name, F1-score, accuracy, hyperparameters, and feature list
POST	/predict	Accepts a single student JSON payload; returns PASS/FAIL, success probability, risk score, risk color, reasons, and interventions
POST	/predict-batch	Accepts a CSV file upload or JSON array for bulk roster predictions
GET	/docs	Auto-generated OpenAPI Swagger UI
GET	/redoc	Auto-generated ReDoc API documentation

8.2 Containerized Deployment

The application is orchestrated via docker-compose.yml, built from a single Dockerfile, running two independent services:

Service	Purpose	Port Mapping
web-app	FastAPI backend + web portal	8000:8000
mlflow-server	MLflow Tracking UI	5000:5000

Required environment variables include PORT (default 8000), with optional overrides for MLFLOW_TRACKING_URI and DATABASE_URL depending on the target deployment environment.

9. Software Requirements Specification and Verification

9.1 Functional Requirements Verification

ID	Requirement	Verification Method
FR-1	Predict student pass/fail outcome and output success probability	/predict endpoint and unit tests
FR-2	Calculate risk score (0–100) and risk level	calculate_risk_metrics() helper
FR-3	Log prediction events asynchronously	monitoring/logger.py — CSV/JSONL output
FR-4	Detect statistical feature drift	monitoring/drift_detection.py + Evidently AI report
FR-5	Automated retraining and candidate evaluation	monitoring/retrain.py

9.2 Unit Testing Summary

All 14 unit tests in the tests/ suite pass with a 100% success rate, covering API endpoints, data-processing utilities, and model target/feature selection logic:

Test / Test Group	Status
test_health_check	PASSED
test_predict_pass_student_with_decision_support	PASSED
test_model_info_endpoint	PASSED
test_metrics_endpoint	PASSED
test_predict_batch_csv_upload	PASSED
TestCreateTarget (3 tests)	PASSED
TestSelectFeatures (3 tests)	PASSED
TestSplitData (3 tests)	PASSED

10. Known Limitations and Future Work

10.1 Limitations Identified and Resolved

- **Out-of-Range Extrapolation:** Tree-based models can produce unreliable predictions when fed feature combinations far outside the training distribution. This was mitigated with educational decision-support guardrails implemented directly in api/main.py, which flag out-of-distribution inputs rather than returning an unqualified prediction.
- **Single-Row Dummy Encoding Drop:** Early single-row dummy encoding with drop_first=True caused missing columns during live inference. Resolved by switching to drop_first=False and explicitly aligning every inference request against the stored preprocessor's feature_columns list, with zero-fill for absent categories.

10.2 Planned Future Work

- Integrate automated Prefect or Airflow DAG scheduling to run drift checks on a daily cadence, rather than relying solely on manual or threshold-triggered retraining.
- Configure a remote cloud object storage backend (AWS S3 or GCS) for DVC, moving dataset and artifact versioning off local disk.
- Implement API key authentication middleware to secure public endpoints and apply per-client rate limiting ahead of any multi-institution rollout.

11. Conclusion and Deployment Summary

The Student Performance Prediction and MLOps System successfully fulfills all functional and operational requirements outlined in the Software Requirements Specification. Model training, decision-support inference, telemetry logging, drift detection, and automated retraining operate as a unified, auditable pipeline rather than disconnected scripts.

The application is fully containerized via Docker and Docker Compose, enabling seamless deployment to local environments, AWS EC2, or managed cloud platforms such as Render. This containerized approach ensures reproducibility of the runtime environment across development, staging, and production.

11.1 Deployment Links

Production Cloud URL: https://student-performance-prediction-with-mlops.onrender.com

Source Code Repository: https://github.com/NITISH1817/The-Pixels.git

11.2 Summary of Outcomes

- An XGBoost champion model achieving 89.1% test accuracy and an F1-score of 0.8268, outperforming Random Forest and Logistic Regression baselines.
- A dual-role FastAPI portal serving both teacher-facing batch analytics and individual student risk calculation.
- A Streamlit MLOps Control Center providing real-time telemetry, Evidently AI drift detection, and automated retraining with a promotion gate.
- A fully verified test suite (14/14 passing) covering API, data-processing, and model-selection logic.
- Containerized, cloud-deployed infrastructure ready for institutional use.